Northwind CRM — Queries

Snowflake SQL. The same files also run unchanged in DuckDB on a laptop. Run them in order: 00 (local only), 01, 02, 03. Source tables in Snowflake: sessions, signups, opportunities. Repo: github.com/bald-phoenix/northwind_assessment

What we had to fix so the data is usable

All fixes happen in 01_clean.sql; 02 and 03 only read the clean tables. Rule ids match the comments in the code and the Data methodology doc.

Rule	Table	Problem found	What we did	Rows
R1	sessions	Same channel written 12 ways ("direct ", "Display ")	Map to 7 channel names	1,284
R2	sessions	135 URL spellings of ~20 pages	Clean the URL, group into 7 page types	all
R3	sessions	Visit with no visitor id	Keep it, mark is_visitor_missing	300
R4	signups	Exact duplicate rows	Remove	66
R5	signups	Fake company sizes (−5, 999,999)	Set to unknown	30
R6	signups	Same lead twice with different details	Keep the most complete row	4 leads
R7	signups	Signup dated July, after the data ends	Use the date of its visit	8
R8	signups	Points to a visit that doesn't exist	Keep it, mark is_session_orphan	25
R9	opportunities	Exact duplicate rows	Remove	18
R10	opportunities	Amount 1,000× the price (35,000 / 110,000)	Divide by 1,000	6
R11	opportunities	$0 deals (nobody pays)	Keep, mark is_zero_amount; never a win	30
R12	opportunities	Negative days_to_close (sign bug)	Recompute from the dates	10
R13	opportunities	Points to a lead that doesn't exist	Keep it, mark is_lead_orphan	15
R14	signups	Country written as the text 'N/A'	Set to unknown	25

00_load_raw.sql — Load the data (local only)

- **What it does:** turns the three CSV files into tables called sessions, signups and opportunities.
- **Why:** in Snowflake these tables already exist, so you skip this file. On a laptop with DuckDB, this step creates them with the same names, so every other query runs unchanged in both places.

```
CREATE OR REPLACE TABLE sessions      AS SELECT * FROM read_csv_auto('data/sessions.csv');
CREATE OR REPLACE TABLE signups       AS SELECT * FROM read_csv_auto('data/signups.csv');
CREATE OR REPLACE TABLE opportunities AS SELECT * FROM read_csv_auto('data/opportunities.csv');
```

01_clean.sql — Clean the tables and connect them

- **What it does:** builds four clean tables, one after the other. Each one uses the one before.
- **Idea 1: LEFT JOIN:** keeps every row on the left even when there's no match on the right. Broken rows get marked instead of silently disappearing.
- **Idea 2: flags are 1/0:** so SUM counts them and AVG gives a rate. is_win is the most important one: 1 = paying customer within 30 days, 0 = not.

STEP 1. sessions_clean — one row per website visit

Fixes channel names (R1), turns 135 messy URLs into 7 page types (R2), marks visits with no visitor (R3).

```
CREATE OR REPLACE TABLE sessions_clean AS
WITH with_clean_url AS (
    SELECT
        s.*,
        -- R2. Make the URL comparable. Read it inside-out:
        --   1) cut everything after "?"         /pricing?plan=starter -> /pricing
        --   2) lowercase                        /HOME -> /home
        --   3) drop the country prefix          /uk/blog/x -> /blog/x
        --   4) drop the trailing slash          /pricing/ -> /pricing
        --   5) if nothing is left, it was "/"   (the home page)
        COALESCE(
            NULLIF(
                REGEXP_REPLACE(
                    REGEXP_REPLACE(LOWER(SPLIT_PART(s.entry_url, '?', 1)),
                                   '^/(uk|in|de|au|ca)(/|$)', '/'),
                    '/+$', ''),
                ''),
            '/') AS url_clean
    FROM sessions s
)
SELECT
    session_id,
    visitor_id,
    CAST(event_dt AS DATE) AS event_dt,

    -- R1. Same channel written different ways ("Display ", "direct") -> one name
    CASE LOWER(TRIM(mktg_channel))
        WHEN 'direct'         THEN 'Direct'
        WHEN 'display'        THEN 'Display'
        WHEN 'email'          THEN 'Email'
        WHEN 'organic search' THEN 'Organic Search'
        WHEN 'paid search'    THEN 'Paid Search'
        WHEN 'referral'       THEN 'Referral'
        WHEN 'social'         THEN 'Social'
    END AS mktg_channel,

    geo,
    device_type,
    entry_url,

    -- R2. Group the clean URL into 7 page types
    CASE
        WHEN url_clean LIKE '/blog%'    OR url_clean LIKE '/resources%' THEN 'content'
        WHEN url_clean IN ('/', '/home', '/index')                      THEN 'home'
        WHEN url_clean LIKE '/pricing%' OR url_clean LIKE '/plans%'     THEN 'pricing'
        WHEN url_clean IN ('/offers/free-crm', '/lp/free-crm')          THEN 'free_crm_lp'
        WHEN url_clean = '/get/starter-trial'                           THEN 'trial_lp'
        WHEN url_clean IN ('/go/crm', '/campaign/spring-2026')          THEN 'campaign_lp'
        WHEN url_clean LIKE '/product%' OR url_clean LIKE '/features%'  THEN 'product'
        ELSE 'other'
    END AS page_type,
```

```
    converted_flag,

    -- R3. 300 visits have no visitor id. Keep them, just mark them.
    CASE WHEN visitor_id IS NULL THEN 1 ELSE 0 END AS is_visitor_missing
FROM with_clean_url;
```

STEP 2. signups_clean — one row per lead

Removes duplicates (R4, R6), blanks out fake values (R5, R14), fixes July dates using the visit date (R7), marks signups whose visit doesn't exist (R8).

```
CREATE OR REPLACE TABLE signups_clean AS
WITH no_duplicates AS (
    -- R4. Remove rows that are exact copies of another row
    SELECT DISTINCT * FROM signups
),
fixed_values AS (
    SELECT
        lead_id,
        visitor_id,
        session_id,
        CAST(created_dt AS DATE) AS created_dt,
        mktg_channel,
        product_code,
        -- R14. The text 'N/A' is not a country -> treat it as unknown
        NULLIF(TRIM(geo), 'N/A') AS geo,
        -- R5. -5 or 999,999 employees are not real -> treat as unknown
        CASE WHEN company_size < 1 OR company_size >= 999999 THEN NULL
             ELSE company_size END AS company_size
    FROM no_duplicates
),
ranked AS (
    -- R6. A few leads appear twice with different details.
    -- Score each row by how complete it is (0, 1 or 2 known fields)
    -- and number the rows of each lead from most to least complete.
    SELECT
        f.*,
        ROW_NUMBER() OVER (
            PARTITION BY lead_id
            ORDER BY (CASE WHEN geo IS NOT NULL THEN 1 ELSE 0 END)
                   + (CASE WHEN company_size IS NOT NULL THEN 1 ELSE 0 END) DESC
        ) AS row_rank
    FROM fixed_values f
)
SELECT
    r.lead_id,
    r.visitor_id,
    r.session_id,

    -- R7. 8 signups are dated in July, after the data ends. Every signup
    -- happens on the same day as its visit, so use the visit date instead.
    CASE WHEN r.created_dt > DATE '2026-06-30' AND s.event_dt IS NOT NULL
         THEN s.event_dt
         ELSE r.created_dt END AS created_dt,
    CASE WHEN r.created_dt > DATE '2026-06-30' AND s.event_dt IS NOT NULL
         THEN 1 ELSE 0 END AS is_date_fixed,

    r.mktg_channel,
    r.product_code,
    CASE r.product_code
        WHEN 'STR'  THEN 'Starter'
        WHEN 'PRO'  THEN 'Pro'
        WHEN 'FCRM' THEN 'Free CRM'
    END AS product,
    r.geo,
```

```
    r.company_size,

    -- R8. 25 signups point to a visit that doesn't exist (broken id).
    -- Keep them, just mark them.
    CASE WHEN s.session_id IS NULL THEN 1 ELSE 0 END AS is_session_orphan
FROM ranked r
LEFT JOIN sessions_clean s
       ON s.session_id = r.session_id
WHERE r.row_rank = 1;          -- keep only the most complete row per lead
```

STEP 3. opportunities_clean — one row per closed deal

Removes duplicates (R9), fixes amounts 1,000× too high (R10), marks $0 deals (R11), recomputes days to close (R12), marks deals whose lead doesn't exist (R13).

```
CREATE OR REPLACE TABLE opportunities_clean AS
WITH no_duplicates AS (
    -- R9. Remove rows that are exact copies of another row
    SELECT DISTINCT * FROM opportunities
)
SELECT
    o.opp_id,
    o.lead_id,
    CAST(o.close_dt AS DATE) AS close_dt,
    o.plan,

    -- R10. 35,000 and 110,000 are the list prices x1000 (unit error)
    CASE WHEN o.amount IN (35000, 110000) THEN o.amount / 1000
         ELSE o.amount END AS amount,

    -- R11. $0 deals closed but nobody pays -> not a paying customer
    CASE WHEN o.amount = 0 THEN 1 ELSE 0 END AS is_zero_amount,

    -- R12. Some days_to_close are negative (sign bug).
    -- Recompute from the dates; if the lead is missing, flip the sign.
    COALESCE(CAST(o.close_dt AS DATE) - g.created_dt,
             ABS(o.days_to_close)) AS days_to_close,

    -- R13. 15 deals point to a lead that doesn't exist (broken id).
    -- Keep them, just mark them.
    CASE WHEN g.lead_id IS NULL THEN 1 ELSE 0 END AS is_lead_orphan
FROM no_duplicates o
LEFT JOIN signups_clean g
       ON g.lead_id = o.lead_id;
```

STEP 4. signup_outcomes — one row per signup, everything we need

to analyze it: where it came from, which SKU, and did it pay The table everything else reads: where the signup came from, which SKU, and is_win.

```
CREATE OR REPLACE TABLE signup_outcomes AS
WITH first_visit AS (
    -- The first day each visitor came to the site
    SELECT visitor_id, MIN(event_dt) AS first_visit_dt
    FROM sessions_clean
    WHERE visitor_id IS NOT NULL
    GROUP BY visitor_id
),
paid_deals AS (
    -- Only deals where someone actually pays
    SELECT * FROM opportunities_clean WHERE is_zero_amount = 0
)
SELECT
```

```
    g.*,

    -- Cohort = the month the lead signed up
    CAST(DATE_TRUNC('month', g.created_dt) AS DATE) AS signup_month,
    CASE WHEN g.created_dt < DATE '2026-04-01' THEN 'Q1' ELSE 'Q2' END AS period,

    -- Where the signup came from (its visit)
    s.device_type,
    s.page_type,
    CASE WHEN fv.first_visit_dt < g.created_dt THEN 'Returning' ELSE 'New' END AS visitor_type,
    CASE
        WHEN g.company_size IS NULL THEN 'Unknown'
        WHEN g.company_size <= 10   THEN '1-10'
        WHEN g.company_size <= 50   THEN '11-50'
        WHEN g.company_size <= 200  THEN '51-200'
        ELSE '201+'
    END AS company_size_bucket,

    -- The deal it turned into, if any
    o.plan   AS won_plan,
    o.amount AS won_amount,
    o.days_to_close,

    -- THE WIN DEFINITION: a paying deal closed within 30 days of signup.
    -- 1 = paying customer, 0 = not. Summing this column counts wins.
    CASE WHEN o.opp_id IS NOT NULL AND o.days_to_close <= 30 THEN 1 ELSE 0 END AS is_win
FROM signups_clean g
LEFT JOIN sessions_clean s ON s.session_id = g.session_id
LEFT JOIN first_visit fv   ON fv.visitor_id = g.visitor_id
LEFT JOIN paid_deals o     ON o.lead_id = g.lead_id;
```

02_funnel.sql — The funnel numbers

- **What it does:** turns signup_outcomes into the tables behind every number in the analysis.
- **Conversion rate:** AVG(is_win). The average of a column of 1s and 0s is the share of 1s, which is the conversion rate.

STEP 1. monthly_funnel — the whole funnel by signup month

visits -> signups -> paying customers -> deal value

```
CREATE OR REPLACE TABLE monthly_funnel AS
WITH visits_by_month AS (
    SELECT CAST(DATE_TRUNC('month', event_dt) AS DATE) AS month,
           COUNT(*) AS sessions
    FROM sessions_clean
    GROUP BY 1
),
signups_by_month AS (
    SELECT signup_month AS month,
           COUNT(*)                    AS signups,
           SUM(is_win)                 AS wins_30d,          -- is_win is 1/0, so SUM counts
wins
           SUM(is_win * won_amount)    AS deal_value_30d     -- amount only counts for wins
    FROM signup_outcomes
    GROUP BY 1
)
SELECT
    month,
    sessions,
    signups,
    wins_30d,
    deal_value_30d,
    ROUND(100.0 * signups  / sessions, 2) AS sess_to_signup_pct,
    ROUND(100.0 * wins_30d / signups, 1)  AS signup_to_win_pct,
    ROUND(100.0 * wins_30d / sessions, 2) AS sess_to_win_pct
FROM visits_by_month
JOIN signups_by_month USING (month)
ORDER BY month;
```

STEP 2. weekly_funnel — same idea, by week (weeks start on Monday)

Only full 7-day weeks: Jan 1-4 and Jun 29-30 are cut off.

```
CREATE OR REPLACE TABLE weekly_funnel AS
SELECT
    CAST(DATE_TRUNC('week', created_dt) AS DATE)            AS week_start,
    COUNT(*)                                                AS signups,
    SUM(CASE WHEN product = 'Free CRM' THEN 1 ELSE 0 END)   AS free_crm_signups,
    SUM(is_win)                                             AS wins_30d,
    ROUND(100.0 * AVG(is_win), 1)                           AS signup_to_win_pct   -- average
of 1/0 = conversion rate
FROM signup_outcomes
GROUP BY 1
HAVING MAX(created_dt) - MIN(created_dt) = 6                -- Monday to Sunday = 6 days apart
ORDER BY 1;
```

STEP 3. product_funnel — the funnel split by SKU, per month

mix_pct = this SKU's share of the month's signups

```
CREATE OR REPLACE TABLE product_funnel AS
SELECT
```

```
    signup_month AS month,
    product,
    COUNT(*)                                    AS signups,
    -- window function: divide by the total signups of the same month
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY signup_month), 1) AS mix_pct,
    SUM(is_win)                                 AS wins_30d,
    ROUND(100.0 * AVG(is_win), 1)               AS signup_to_win_pct,
    SUM(is_win * won_amount)                    AS deal_value_30d
FROM signup_outcomes
GROUP BY signup_month, product
ORDER BY month, product;
```

STEP 4. slice_conversion — conversion for every origin slice

Trick: stack the table 6 times (one per dimension) with UNION ALL, so a single GROUP BY gives every slice at once.

```
CREATE OR REPLACE TABLE slice_conversion AS
WITH stacked AS (
    SELECT signup_month, period, is_win, product, 'channel'      AS dimension, mktg_channel
AS value FROM signup_outcomes
    UNION ALL
    SELECT signup_month, period, is_win, product, 'entry_page'   AS dimension, page_type
AS value FROM signup_outcomes
    UNION ALL
    SELECT signup_month, period, is_win, product, 'device'       AS dimension, device_type
AS value FROM signup_outcomes
    UNION ALL
    SELECT signup_month, period, is_win, product, 'visitor_type' AS dimension, visitor_type
AS value FROM signup_outcomes
    UNION ALL
    SELECT signup_month, period, is_win, product, 'company_size' AS dimension,
company_size_bucket AS value FROM signup_outcomes
    UNION ALL
    SELECT signup_month, period, is_win, product, 'geo'          AS dimension, COALESCE(geo,
'Unknown') AS value FROM signup_outcomes
)
SELECT
    dimension,
    value,
    signup_month AS month,
    period,
    COUNT(*)                                                          AS signups,
    SUM(is_win)                                                       AS wins_30d,
    ROUND(100.0 * AVG(is_win), 1)                                     AS signup_to_win_pct,
    ROUND(100.0 * AVG(CASE WHEN product = 'Free CRM' THEN 1 ELSE 0 END), 1) AS
free_crm_share_pct
FROM stacked
WHERE value IS NOT NULL            -- the 25 signups with no visit have no page/device
GROUP BY dimension, value, signup_month, period
ORDER BY dimension, value, month;
```

STEP 5. finance_view — customers by the month the deal CLOSED

(how finance counts). Includes the 15 deals with a broken lead id.

```
CREATE OR REPLACE TABLE finance_view AS
SELECT
    CAST(DATE_TRUNC('month', close_dt) AS DATE)            AS close_month,
    SUM(CASE WHEN is_zero_amount = 0 THEN 1 ELSE 0 END)    AS paying_customers,
    COUNT(*)                                               AS all_closed_opps,
    SUM(amount)                                            AS deal_value
FROM opportunities_clean
```

```
GROUP BY 1
ORDER BY 1;
```

03_decomposition.sql — Why the number of paying customers changed

- **The question:** paying customers went from 316 to 168 per month. How much of that is because of more signups, a different SKU mix, or worse conversion inside each SKU?
- **The idea:** wins = signups × conversion, and conversion = each SKU's share × its own conversion. Change one thing at a time, holding the rest at the average of both periods, and the three pieces add up exactly to the total change.
- **The answer:** volume +56, mix −201, rate −3 paying customers per month. Almost all of the drop is the SKU mix.
- **The one assumption:** Free CRM didn't exist in Q1, so it has no Q1 conversion. We use its Q2 conversion for both periods, so its whole effect counts as mix.

```
CREATE OR REPLACE TABLE kitagawa AS
WITH sku_by_period AS (
    -- Average signups and wins per month, for each period and SKU
    SELECT period,
           product,
           COUNT(*) / 3.0    AS signups_mo,     -- 3 months per period
           SUM(is_win) / 3.0 AS wins_mo
    FROM signup_outcomes
    GROUP BY period, product
),
all_combinations AS (
    -- Every SKU in both periods, so Free CRM gets a Q1 row (with zeros)
    SELECT p.period, k.product
    FROM (VALUES ('Q1'), ('Q2')) AS p(period)
    CROSS JOIN (SELECT DISTINCT product FROM signup_outcomes) AS k
),
filled AS (
    SELECT a.period,
           a.product,
           COALESCE(b.signups_mo, 0) AS signups_mo,
           COALESCE(b.wins_mo, 0)    AS wins_mo
    FROM all_combinations a
    LEFT JOIN sku_by_period b
           ON b.period = a.period AND b.product = a.product
),
shares_and_rates AS (
    SELECT period,
           product,
           signups_mo / SUM(signups_mo) OVER (PARTITION BY period) AS mix,    -- SKU share of
signups
           wins_mo / NULLIF(signups_mo, 0)                         AS rate    -- SKU conversion
(NULL if no signups)
    FROM filled
),
side_by_side AS (
    -- Q1 and Q2 next to each other, one row per SKU
    SELECT q1.product,
           q1.mix                       AS mix_q1,
           q2.mix                       AS mix_q2,
           COALESCE(q1.rate, q2.rate)   AS rate_q1,     -- the Free CRM assumption
           q2.rate                      AS rate_q2
    FROM shares_and_rates q1
    JOIN shares_and_rates q2
      ON q2.product = q1.product
    WHERE q1.period = 'Q1' AND q2.period = 'Q2'
),
period_totals AS (
    SELECT period,
           SUM(signups_mo)                  AS n,       -- signups per month
           SUM(wins_mo)                     AS w,       -- wins per month
           SUM(wins_mo) / SUM(signups_mo)   AS r        -- blended conversion
```

```
    FROM filled
    GROUP BY period
),
t AS (
    SELECT q1.n AS n1, q2.n AS n2, q1.w AS w1, q2.w AS w2, q1.r AS r1, q2.r AS r2
    FROM period_totals q1
    JOIN period_totals q2 ON q1.period = 'Q1' AND q2.period = 'Q2'
)
SELECT 'Q1 wins / month' AS step, NULL AS product, w1 AS wins FROM t
UNION ALL
SELECT 'Volume effect', NULL, (n2 - n1) * (r1 + r2) / 2 FROM t
UNION ALL
SELECT 'Mix effect', s.product,
       (t.n1 + t.n2) / 2 * (s.mix_q2 - s.mix_q1) * (s.rate_q1 + s.rate_q2) / 2
FROM side_by_side s CROSS JOIN t
UNION ALL
SELECT 'Rate effect', s.product,
       (t.n1 + t.n2) / 2 * (s.mix_q1 + s.mix_q2) / 2 * (s.rate_q2 - s.rate_q1)
FROM side_by_side s CROSS JOIN t
UNION ALL
SELECT 'Q2 wins / month', NULL, w2 FROM t;


-- The waterfall: one bar per effect. Volume + Mix + Rate = Q2 - Q1, exactly.
SELECT step, ROUND(SUM(wins), 1) AS wins_per_month
FROM kitagawa
GROUP BY step
ORDER BY CASE step WHEN 'Q1 wins / month' THEN 1
                   WHEN 'Volume effect'   THEN 2
                   WHEN 'Mix effect'      THEN 3
                   WHEN 'Rate effect'     THEN 4
                   ELSE 5 END;
```